

BMMS — Modernization & Technology Upgrade Recommendations

Analysis Based on the 2010–2011 Codebase and Verified Database Schema

Context Before Recommending Anything

Two facts should shape how this document is used:

1. **The stack is genuinely from its era.** ASP.NET Web Forms, .NET Framework, `System.Web.Mail`, `AjaxControlToolkit`, and `FreeTextBox` were all mainstream choices in 2010–2011 and are now legacy: Web Forms has no future version, classic ASP.NET (`System.Web`) doesn't run on ASP.NET Core, and several referenced libraries (`FreeTextBox`, `System.Web.Mail`) are unmaintained.
2. **Lakshmi Vilas Bank, the institution this software was built for, no longer exists as an independent entity.** It was an Indian private sector bank that became defunct on 27 November 2020, when it was amalgamated into DBS Bank India Limited under Section 45 of the Banking Regulation Act, with the RBI moratorium lifted and the merger taking effect that same day. If this system is still in use anywhere, it would now sit inside DBS Bank India's IT environment (with DBS's own governance-software standards likely applying), or it may be running only as a legacy/archival system. Worth confirming which situation applies before investing in a full modernization — the recommendations below are organized so that the highest-value items (security hardening) are worth doing regardless, while the larger re-platforming work only makes sense if the software has an ongoing future.

Recommendations below are grounded in what was actually found in the source code and the verified database schema (see the earlier documentation and SQL structure reports) — not generic modernization boilerplate.

Priority Framework

Tier	Meaning	Timeframe
 Critical	Security or data-integrity gaps confirmed in this codebase; fix regardless of any other decision	Immediate
 High	Materially reduces risk or technical debt; feasible without a full rewrite	1–2 quarters
 Medium	Meaningful modernization; best done as part of a planned re-platform	6–18 months
 Strategic	Full re-platform / architecture change	18+ months, only if the system has a long-term future

 **Critical — Fix Regardless of Any Modernization Decision**

These were confirmed directly in the code and schema, not assumed:

1. **Password storage.** `Common.EncryptVal / DecryptVal` uses reversible (Base64-style) encryption for login passwords and the SMTP password — not a salted hash. Anyone with database access recovers every password in plaintext. **Replace with a proper password hash** (bcrypt, Argon2id, or PBKDF2 with a per-user salt) immediately; this requires a forced password reset for all users on migration since the old values can't be converted.
2. **Plaintext SQL credentials in** `Web.config (uid=sa;password=sa)`. Move to a secrets manager (Azure Key Vault, AWS Secrets Manager, or at minimum encrypted configuration sections) and stop using the `sa` account — create a least-privilege application login.
3. **Seven of twelve delete stored procedures have no reference-integrity check at all** (`spr_DeleteCommitteeByCommitteeId`, `spr_DeleteCommitteeMemberByCommitteeMemberId`, `spr_DeleteDepartmentByDeptId`, `spr_DeleteDesignationByDesignationId`, `spr_DeleteMemberTypeByMemberTypeId`, `spr_DeleteSubjectTypeBySubjectTypeId`, `spr_DeleteDecisionTypeByDecisionTypeId`) — confirmed by reading the actual procedure bodies. These can silently orphan child data. Add the same `IF (SELECT COUNT(*) ...) <= 0` pattern already used correctly in `spr_DeleteMemberByMemberId` and `spr_DeleteMeetingByMeetingId`, or better, add real foreign key constraints with `ON DELETE RESTRICT` and let SQL Server enforce it.
4. `spr_DeleteUserByUserId` **has no check either** — a user with an approval history, created agenda items, or audit trail entries can be deleted outright, breaking every FK-less reference to that `UserId` throughout the schema.
5. **No enforced foreign keys on several relationship columns that the application clearly treats as relationships:** `Agenda.CommitteeId`, `Users.ManagerId`, `Member.SexId`, `Member.State_Id`, `Meeting.ChairmanNameId`, `Action_Item.MinutesId`, `Alert_User_Map.UserId`, `ActionItem_User_Map.UserId`. Add the missing constraints — this is what's letting the delete bugs above go unnoticed by SQL Server itself.
6. `Source_Type` and `Audit_Trail_SendMail` **have no primary key at all**. Add one to each; a table with no PK can silently accumulate duplicate rows and is invisible to most replication/CDC/ORM tooling.
7. **Authorization is UI-only** (`Session["RoleCode"]` checks scattered per page, no central policy). Any page reachable by a guessed URL is only as safe as that page's own code-behind. This is a real risk in a governance/compliance system where role separation (who can approve vs. who can enter) is the entire point.

None of these require a framework change — they're fixable inside the current ASP.NET Web Forms + stored-procedure architecture, and should be treated as a pre-modernization cleanup sprint regardless of what happens next.

● High — Feasible Without a Full Rewrite

Area	Current state	Recommendation
Session/auth model	Session-based, <code>RoleCode</code> string checks per page	Move to ASP.NET's built-in <code>[Authorize(Roles=...)]</code> (available even in Web

		Forms via <code>web.config</code> (<code><location></code> path-based authorization rules) as an intermediate step before a full auth overhaul
Mail	Legacy <code>System.Web.Mail</code>	Swap to <code>System.Net.Mail</code> (drop-in-ish replacement, still works on .NET Framework) or a transactional email API (SendGrid/Amazon SES) with proper deliverability, bounce handling, and TLS
PDF generation	iTextSharp (the free version used here was later relicensed AGPL from v5 onward — worth checking exactly which version is referenced, since AGPL has real commercial-use implications)	Migrate to QuestPDF (MIT-licensed, modern .NET) or iText 7 under a proper commercial license if staying with iText
Rich text editor	FreeTextBox (unmaintained since ~2010)	Replace with TinyMCE or CKEditor 5 — both have modern, actively maintained ASP.NET-compatible integrations
File storage	Hardcoded local paths (<code>C:\board\photo.jpg</code> , <code>C:\bmms\bmms.doc</code> , backup to <code>C:\BMMS\Site\Files\SqlBackup\</code>)	Move to blob storage (Azure Blob Storage / S3-compatible) — local paths break the moment this runs on more than one server or in a container, and they're a backup/DR liability as-is
Database backup	Manual on-demand <code>spr</code> + SMO trigger from the web app	Move to SQL Server Agent scheduled jobs + offsite backup, decoupled from the web application process — a web app should not be the thing responsible for triggering its own database backups
Meeting time storage	<code>Meeting.MeetingTime</code> is <code>varchar(10)</code>	Change to a proper <code>time</code> (or combine into a single <code>datetime2</code> with

		MeetingDate) — this single change would eliminate the 12/24-hr formatting bug at its root instead of patching it in every report
"Reset Database"	Truncates only tagging/attendance/audit tables, silently leaves Committee/Meeting/Agenda/Minutes/Member/Users untouched	Either fix the procedure to match its documented behavior, or rename/relabel it in the UI so admins aren't misled about what it actually does
Audit trail	Three separately-named, inconsistent tables (Audit_Trail_Master, Audit_Trail_SqlBackup, Audit_Trial_Login — note the schema-level typo) plus an undiscovered fourth (Audit_Trail_SendMail)	Consolidate into a single, consistently-named audit schema; fix the "Trial"/"Master"("Maser") typos as part of a proper migration rather than living with them indefinitely
Testing	Zero automated tests in the solution	Add a test project (xUnit or NUnit) starting with the BAL layer — AgendaMgr, MinutesMgr -equivalent logic, and the encryption/decryption helpers are the highest-value places to start, since they're pure business logic with no UI dependency

● Medium — Best Done as Part of a Planned Re-platform

Data access layer

Replace the hand-rolled *DB / *DAL classes and raw ADO.NET + stored-procedure calls with:

- **Entity Framework Core** (if moving to .NET 8+/ASP.NET Core) for standard CRUD, keeping stored procedures only for genuinely complex reporting queries, or
- **Dapper** as a lighter-weight micro-ORM if the team wants to keep writing SQL by hand but eliminate the boilerplate DataTable to-object mapping currently duplicated across ~20 DAL classes.

Either choice also naturally forces the missing-FK problems above to surface at compile/migration time rather than silently at runtime.

Frontend

- Web Forms' server-side controls, ViewState, and full-page postbacks are the single biggest legacy liability — there is no incremental "patch" for this; it requires rewriting each page.
- Recommended target: **ASP.NET Core MVC or Razor Pages** with a modern component layer, or **Blazor Server** if the team wants to stay in C# end-to-end and the current developer base has limited JavaScript/TypeScript depth. A full SPA (React/Angular/Vue) is the right call only if there's an appetite to also build a proper REST/JSON API layer — which is worth doing anyway if a mobile app for board members is ever wanted (a very common ask for governance software, since board members increasingly expect to review agenda packs on tablets).
- The `ddllevelsmenu`-style custom dropdown JS and inline role-based menu rendering (`MenuUserControl.ascx`) should become policy-driven navigation generated from the same authorization system, not duplicated logic.

Database platform

- The schema was scripted from a SQL Server instance at **compatibility level 100** (SQL Server 2008) — confirmed directly in the script header. Even without a full re-platform, upgrading to a current SQL Server version (or Azure SQL Database / Managed Instance) and raising the compatibility level unlocks query-store diagnostics, better execution plans, and `Always Encrypted` for the PII columns (`Member.DOB`, `Address1-3`, `EmailId`, `DinNo`) that are currently stored as plain `varchar` with no column-level encryption.
- Introduce a real migrations tool (EF Core Migrations, DbUp, or Flyway) so schema changes are versioned and repeatable instead of ad hoc SSMS scripts.

Reporting

- Current reports are single-purpose PDF generators (`PDF_Reports.cs`) with no interactivity. Consider a proper reporting/BI layer (Power BI embedded, or even a lightweight in-app dashboard) for governance analytics that the bank's board would plausibly want: attendance trends per director, sitting-fee spend over time, agenda approval turnaround time, followup-item aging — none of which the current system surfaces, even though the underlying data (`Attendance`, `Committee_Member.ApplicableFee`, `Agenda` approval timestamps, `Action_Item.IsComplied`) already supports it.

Strategic — Full Re-platform Considerations

Only pursue this tier if the software has a genuine ongoing future (see the DBS/LVB context note above) — otherwise the  and  tiers deliver most of the risk reduction at a fraction of the cost.

1. **Target framework:** .NET 8/9 (LTS), cross-platform, containerizable, with dramatically better performance than .NET Framework 4.x.
2. **Architecture:** move from the current "UI project → BAL project → DAL project" 3-tier layering (which is really just a folder-organization pattern, not true separation) toward a Clean Architecture / Vertical Slice structure with dependency injection (built into ASP.NET Core), so business rules like the agenda approval workflow and the Followup/Action Item lifecycle are testable independent of both the UI and the database.
3. **Identity & access:** ASP.NET Core Identity or an external identity provider (Azure AD / Entra ID / Okta)

- ...replacing `Session["RoleCode"]` string checks — with **claims-based, policy-driven authorization** replacing the `Session["RoleCode"]` string checks — this also enables proper **multi-factor authentication**, which a board-minutes system handling sensitive, price-sensitive corporate information should have as a baseline, not an afterthought.
4. **Real-time features:** replace the current "refresh the dashboard to see new alerts" model with **SignalR** push notifications for agenda approvals, new action items, and meeting reminders.
 5. **Digital signatures / e-sign:** the manual describes a fully manual "In Attendance" signatory model on printed minutes. Modern board-management platforms (Diligent, BoardEffect, and similar commercial products this system directly competes with conceptually) universally offer e-signature integration for minutes confirmation — worth evaluating DocuSign/Adobe Sign integration or India-specific digital signature certificate (DSC) support if board resolutions need to remain legally executable digitally, which is increasingly the norm and expectation for Indian corporate governance.
 6. **Containerization & CI/CD:** Docker images, a proper pipeline (GitHub Actions/Azure DevOps) running the test suite from the 🟠 tier on every commit, and infrastructure-as-code (Terraform/Bicep) for repeatable environment provisioning — none of which exists today.
 7. **Observability:** structured logging (Serilog/Application Insights) replacing the current `Utilities.WriteErrorLog` -to-flat-file pattern, plus proper application performance monitoring.
 8. **Full-text/document search:** `Agenda.AgendaText` and `Minutes.ActualResolution` are stored as unindexed `varchar(max)` — for an organization with years of board history, full-text search (SQL Server Full-Text Search at minimum, or Azure Cognitive Search / Elasticsearch for a richer experience) would make historical minutes actually discoverable rather than requiring manual browsing by meeting number.
-

Suggested Sequencing

- Phase 0 (immediate, weeks): 🟡 Critical security & data-integrity fixes
– no framework change, done in the existing codebase
- Phase 1 (1–2 quarters): 🟠 High-value swaps: mail, PDF library, rich-text editor, file storage → blob storage, add a test project, fix `MeetingTime` data type, fix Reset Database behavior